Code and Branch Coverage Report

Online Exam Portal | Software Engineering Mini-Project, Part 3 | 6 November 2026

Prepared for Chiranthan V | PES2UG24CS138 | Section C | Test lead

Tested PR head: fa38c385bd64c86603fa4970db873971b994eaf8

GitHub executed merge: ee9b05cf0ee801885530630d0f4e6af7c6d3462d. Its file tree equals the PR head (verified with git rev-parse).

Report-only commits may follow; they do not change the application measured in this report. Exact historical operational revisions are stated separately.

1. Measured scope and result

Backend statement coverage: 1,099 / 1,144 = 96.07%. Backend branch coverage: 213 / 252 = 84.52%. The coverage.py combined statement+branch metric is (1,099+213)/(1,144+252) = 93.98%, displayed as 94% by its console report. These metrics must not be interchanged.

Core scope services.py + assessment.py: 65/67 statements = 97.01%; 28/30 branches = 93.33%; combined 95.88%. The configured 70% core combined gate and 85% whole-scope combined gate both passed. No separate 85% branch-only gate is configured; 84.52% branch coverage must not be rounded up to claim that.

2. Instrumentation and denominator

coverage.py 7.17.0, branch=true, executed with 100 PostgreSQL backend tests on the named GitHub merge revision. Configured source is backend/portal. Migrations, tests and management commands are omitted. Django configuration, scripts, frontend JavaScript, third-party dependencies and container files are outside this coverage denominator. Monitor/browser execution results are separate; frontend statement/branch coverage was not collected.

A statement means an executable source line counted by coverage.py, not every physical line. Branches are instrumented possible exits/arcs, not a proof of every Boolean combination or MC/DC. Zero-denominator modules are shown N/A in the tables rather than inventing a measured 100% result.

3. Statement coverage by module

Module	Statements	Covered	Missing	Coverage
__init__.py	0	0	0	N/A (empty)
assessment.py	20	19	1	95.00%
assessment_views.py	186	171	15	91.94%
auth_views.py	156	156	0	100.00%
authentication.py	8	8	0	100.00%
models.py	198	198	0	100.00%
notifications.py	11	11	0	100.00%
permissions.py	7	7	0	100.00%
roster.py	81	75	6	92.59%
serializers.py	98	97	1	98.98%
services.py	47	46	1	97.87%
views.py	332	311	21	93.67%

TOTAL: 1,144 statements; 1,099 covered; 45 missing; 96.07% statement coverage. Core engine statement coverage is 65/67 = 97.01%. Coverage is evidence of execution, not completeness of assertions or absence of defects.

Evidence and reproduction

Backend CI: https://github.com/chiranthanv/online-exam-portal/actions/runs/42198702219

Frontend/browser CI: https://github.com/chiranthanv/online-exam-portal/actions/runs/42198702163

Permanent repository evidence: docs/testing/part3/evidence/2026-11-06/. The manifest records SHA-256 hashes; original backend ZIP was verified against GitHub's artifact digest before extraction. Other original ZIP digests are retained in the manifest; selected raw JSON/text files are preserved without report screenshots.

Detailed coverage: coverage.json, coverage.xml and backend-coverage-original.zip (includes the interactive htmlcov/index.html report). Backend log lists every test; frontend JSON lists each assertion/scenario and actual status.

Part B - Branch Coverage Report

1. Measured scope and result

Backend statement coverage: 1,099 / 1,144 = 96.07%. Backend branch coverage: 213 / 252 = 84.52%. The coverage.py combined statement+branch metric is (1,099+213)/(1,144+252) = 93.98%, displayed as 94% by its console report. These metrics must not be interchanged.

Core scope services.py + assessment.py: 65/67 statements = 97.01%; 28/30 branches = 93.33%; combined 95.88%. The configured 70% core combined gate and 85% whole-scope combined gate both passed. No separate 85% branch-only gate is configured; 84.52% branch coverage must not be rounded up to claim that.

2. Instrumentation and denominator

coverage.py 7.17.0, branch=true, executed with 100 PostgreSQL backend tests on the named GitHub merge revision. Configured source is backend/portal. Migrations, tests and management commands are omitted. Django configuration, scripts, frontend JavaScript, third-party dependencies and container files are outside this coverage denominator. Monitor/browser execution results are separate; frontend statement/branch coverage was not collected.

A statement means an executable source line counted by coverage.py, not every physical line. Branches are instrumented possible exits/arcs, not a proof of every Boolean combination or MC/DC. Zero-denominator modules are shown N/A in the tables rather than inventing a measured 100% result.

3. Branch coverage by module

Module	Exits	Covered	Missing	Partial	Coverage
__init__.py	0	0	0	0	N/A (no branches)
assessment.py	10	9	1	1	90.00%
assessment_views.py	64	49	15	13	76.56%
auth_views.py	16	16	0	0	100.00%
authentication.py	2	2	0	0	100.00%
models.py	2	2	0	0	100.00%
notifications.py	4	4	0	0	100.00%
permissions.py	0	0	0	0	N/A (no branches)
roster.py	34	28	6	6	82.35%
serializers.py	26	25	1	1	96.15%
services.py	20	19	1	1	95.00%
views.py	74	59	15	15	79.73%

TOTAL: 252 possible exits, 213 covered, 39 missing. Thirty-seven partially covered branch locations are reported. Partial-branch locations and missing exits are different counts; do not add them together. Core engine branch coverage is 28/30 = 93.33%.

4. Uncovered source and branch exits

Module	Missing executable lines	Missing exits (source -> destination)
assessment.py	18	17->18
assessment_views.py	74, 75, 98, 123, 148, 190, 194, 195, 196, 242, 244, 251, 261, 264, 324	97->98, 122->123, 147->148, 189->190, 193->194, 195->196, 195->197, 210->212, 241->242, 243->244, 246->251, 254->256, 260->261, 263->264, 323->324
roster.py	34, 39, 41, 48, 76, 80	33->34, 38->39, 40->41, 47->48, 75->76, 79->80
serializers.py	155	154->155
services.py	50	49->50
views.py	103, 104, 131, 210, 213, 349, 350, 423, 431, 432, 436, 451, 456, 458, 460, 513, 514, 516, 518, 524, 525	52->57, 128->131, 209->210, 212->213, 303->312, 348->349, 422->423, 435->436, 450->451, 455->456, 457->458, 459->460, 515->516, 517->518, 521->529

The JSON is the authoritative machine-readable list of missing lines and arcs. Negative arc destinations, if present, identify function exits in coverage.py. Missing coverage is not automatically a product defect; it identifies unexercised behavior for focused review.

Examples: assessment.py line 18/exit 17->18 is the explicit persisted-answer selected_option_id=None guard; current unanswered fixtures also exercise the absence-of-answer route. services.py line 50/exit 49->50 is the exam-total overflow rejection. Additional unexercised roster/permission/error paths remain visible above. No synthetic tests were added merely to increase the percentage, and 100% branch coverage is not claimed.

5. Reproduction and audit checks

```
coverage run backend/manage.py test portal.tests --verbosity 2
coverage report
coverage report --include=backend/portal/services.py,backend/portal/assessment.py --fail-under=70
coverage json -o coverage.json
coverage xml
coverage html
```

Run from repository root using requirements-dev.txt, Python3.11, DEBUG=1 and a disposable PostgreSQL16 DATABASE_URL. pyproject.toml supplies branch/source/omit settings. Open htmlcov/index.html from the original backend artifact to inspect highlighted source. Raw CI JSON/XML and the original archive are retained, avoiding reliance on rounded console text.

Local Windows/SQLite cross-check discovered 100 tests: 95 passed, 5 PostgreSQL-specific skips, no failures. Its coverage is retained separately and not merged into the authoritative PostgreSQL denominator. Earlier invocation failed before tests because the reporting wrapper did not import coverage.cmdline explicitly; corrected invocation and successful results are retained. This setup error was not a product test failure.

Evidence and reproduction

Backend CI: https://github.com/chiranthanv/online-exam-portal/actions/runs/42198702219

Frontend/browser CI: https://github.com/chiranthanv/online-exam-portal/actions/runs/42198702163

Permanent repository evidence: docs/testing/part3/evidence/2026-11-06/. The manifest records SHA-256 hashes; original backend ZIP was verified against GitHub's artifact digest before extraction. Other original ZIP digests are retained in the manifest; selected raw JSON/text files are preserved without report screenshots.

Detailed coverage: coverage.json, coverage.xml and backend-coverage-original.zip (includes the interactive htmlcov/index.html report). Backend log lists every test; frontend JSON lists each assertion/scenario and actual status.